UPTAEB · SECCIÓN IN3103 · EQUIPO 8

Automatización de Pruebas y Pipelines de Software

"Software probado por máquinas: más rápido, más confiable y sin errores humanos."

CASO PRÁCTICO:
Galletas Naty's

DEMO EN VIVO:
Carga y Estrés con Apache JMeter

PRESENTACIÓN

Agenda de Exposición y Ponentes

Estructura de la defensa técnica dividida en 5 módulos especializados

1	**Edgmairys Apostol**	**Fundamentos de Automatización** Concepto, Manual vs Auto, ROI y Pirámide de Pruebas
2	**Moisés Pereira**	**Definición y Tipos de Pipelines** Tuberías CI/CD, Triggers y Anatomía en GitHub Actions
3	**Elías Armas**	**Diseño de Scripts y JMeter** Patrón AAA, Modularidad, POM y Arquitectura JMeter
4	**Ismael Torres**	**Herramientas y Demo en Vivo** Ecosistema de Calidad, Simulación de Carga y Dashboards
5	**Santiago Loyo**	**Estrategias CI/CD y Despliegue** Shift-Left, Quality Gates, Blue/Green y Time-to-Market

FUNDAMENTOS

¿Qué es la Automatización de Pruebas?

Definición formal, objetivos del QA moderno y pilares de eficiencia

CONCEPTO ESENCIAL

Uso de programas, scripts y herramientas especializadas para ejecutar casos de prueba de forma automática, comparando los resultados reales con los esperados y reportando si pasa o falla sin intervención humana continua.

OBJETIVO REAL:
No es eliminar las pruebas manuales al 100%, sino liberar tiempo de lo repetitivo para enfocarse en lógica compleja y experiencia de usuario.

Ahorra Tiempo

Pruebas que tomaban horas se ejecutan en segundos de manera desatendida.

Menos Bugs

Detección temprana en desarrollo: 10x más barato que resolverlo en producción.

100% Reutilizable

El mismo script se ejecuta exactamente igual en cada nueva versión del sistema.

Mayor Calidad

Confiabilidad absoluta en procesos críticos: compras, pagos e inventario.

FUNDAMENTOS

Pruebas Manuales vs. Automatizadas & Criterio de ROI

Comparativa técnica directa y evaluación del Retorno de Inversión

CRITERIO	PRUEBAS MANUALES	PRUEBAS AUTOMATIZADAS
Velocidad	Lenta y secuencial	**Ultrarrápida y en paralelo**
Repetición	Cansa al tester y provoca fallos	Siempre idéntica y consistente
Costo	Alto y acumulativo a largo plazo	**Alto al inicio, bajo después (ROI)**
Ideal para	Exploración, usabilidad visual y UX	**Regresión, APIs, estrés y carga masiva**

SÍ AUTOMATIZAR CUANDO:

- Tareas repetitivas que se ejecutan en cada versión.
- Pruebas de estrés y alta concurrencia (JMeter).
- Flujos críticos de negocio (login, carrito, pagos).

NO VALE LA PENA AUTOMATIZAR CUANDO:

- La interfaz cambia a diario (scripts se rompen).
- Pruebas de usabilidad subjetiva o percepción visual.
- Pruebas de una sola vez (se gasta más tiempo programando).

La Pirámide de Automatización de Pruebas

Estructura arquitectónica profesional según velocidad, costo y cobertura



Pruebas Unitarias (Base Ancha) · [PHPUnit]

Prueban funciones y métodos aislados del código fuente sin tocar navegador. Son cientos, ultraveloces (milisegundos) y muy económicas.

Pruebas de Integración y API (Centro) · [Postman / Endpoints]

Verifican que los módulos conversen correctamente con la base de datos MySQL y servicios externos.

Pruebas de Interfaz UI (Cúspide) · [Selenium / Cypress]

Simulan la interacción del usuario final en el navegador. Son más lentas y vulnerables a cambios de diseño.

PIPELINES

¿Qué es un Pipeline? La Tubería de Software

El corazón operativo de la filosofía DevOps y su flujo automatizado

CONCEPTO TÉCNICO

Un Pipeline es una serie de pasos automatizados que toman el código fuente desde la laptop del desarrollador (PHP/MySQL/XAMPP) y lo llevan de forma segura hasta el servidor de producción.

LA ANALOGÍA DE GALLETAS NATYS:

Igual que en una fábrica: la masa entra por un extremo, pasa por moldeado, horneado, control de calidad y sale empaquetada. El pipeline hace exactamente lo mismo con cada línea de código.



PIPELINES

Tipos de Pipelines & Ejemplo en GitHub Actions

Diferencias entre CI y CD y configuración práctica en formato YAML

CI (Integración Continua)

Cada commit se valida, compila y prueba automáticamente. Si falla, el cambio no entra a la rama principal.

CD (Continuous Delivery vs Deployment)

Delivery: Listo para producción con un clic humano.
Deployment: Despliegue 100% automático al pasar tests.

Triggers (Los Disparadores)

Eventos que encienden el pipeline: un git push, un Pull Request o triggers programados de madrugada (cron).

YAML · .github/workflows/ci.yml (Galletas Natys)

```yaml
name: Pipeline Galletas Natys

on: push  # Trigger: cada commit

jobs:
  pruebas-calidad:
    runs-on: ubuntu-latest
    steps:
      - name: Descargar repositorio
        uses: actions/checkout@v4

      - name: Instalar dependencias
        run: composer install

      - name: Ejecutar pruebas unitarias
        run: vendor/bin/phpunit
        # ¡Si falla, detiene el despliegue!
```

DISEÑO DE SCRIPTS

Arquitectura y Buenas Prácticas en Scripts de Prueba

Patrón universal AAA (Arrange, Act, Assert) y principios de ingeniería

¿QUÉ ES UN SCRIPT DE PRUEBA?

Un guion de código o configuración que instruye a la herramienta (JMeter, PHPUnit) a realizar acciones exactas, enviar datos a la tienda de Natys y comprobar si el servidor respondió correctamente con código HTTP 200.

1. PREPARAR (Arrange)

Setup & Pre-conditions

Se inicializan variables, credenciales del usuario y conexiones limpias a la base de datos MySQL.

2. ACTUAR (Act)

Execution / Acción

Se realiza la acción a probar (ej. enviar una petición POST al carrito con 5 cajas de galletas).

3. VERIFICAR (Assert)

Assertions / Checkpoints

Corazón de la prueba: comprueba si la respuesta fue 200 OK y si el stock en la BD se descontó en 5.

PATRONES DE DISEÑO PROFESIONAL:

- Modularidad e Independencia: Scripts que no dependen entre sí y reutilizan funciones comunes (ej. login).
- Data-Driven Testing (DDT): Mismo script alimentado por lotes de datos desde un archivo CSV con múltiples clientes.
- Page Object Model (POM): Desacopla la interfaz visual de la lógica de prueba para facilitar el mantenimiento.

DISEÑO DE SCRIPTS

Plan de Pruebas en Apache JMeter

Anatomía técnica de un script de carga y estrés aplicado a Galletas Natys

Test Plan	Elemento raíz global del proyecto de prueba
Thread Group	Usuarios virtuales concurrentes simulados (50 a 100 hilos)
HTTP Sampler	Petición web específica (GET /catalogo, POST /carrito)
Response Assertion	Regla de verificación: código 200 OK y confirmación
Listeners	Generador de reportes dinámicos, latencia y tablas

CARACTERÍSTICAS DE JMETER

- Open Source en Java: Sin costo de licencias.
- Multiprotocolo: HTTP, HTTPS, JDBC, SOAP, REST.
- Multihilo masivo: Simula miles de clientes simultáneos.
- Reportes Dashboard: Genera gráficos interactivos en HTML.
- Fácil integración: Se ejecuta por consola en CI/CD.

EN GALLETAS NATYS: Permite evaluar cómo responde Apache y MySQL antes de lanzar una promoción comercial.

HERRAMIENTAS Y DEMO

Herramientas del Ecosistema de Calidad

Cada herramienta cubre una fase específica de la tubería continua de software

Apache JMeter

Carga, Rendimiento y Estrés

La herramienta estrella: simula concurrencia masiva de usuarios y mide latencia.

PHPUnit

Pruebas Unitarias Backend

Valida funciones matemáticas del carrito de compras y reglas de inventario en PHP.

Postman / Newman

Pruebas de APIs y Endpoints

Comprueba peticiones HTTP a la base de datos MySQL tanto en GUI como en consola.

Selenium / Cypress

Pruebas de Interfaz (UI)

Automatiza clics, navegación y formularios directamente en el navegador web.

GitHub Actions

Orquestador CI/CD en la Nube

Ejecuta automáticamente los scripts al hacer push mediante archivos YAML.

Jenkins / Stack LAMP

Servidor Local y Entorno Base

Servidor de integración continua clásico y entorno base (Linux, Apache, MySQL, PHP).

HERRAMIENTAS Y DEMO

Caso Práctico: Tienda Web Galletas Natys

Simulación de carga con JMeter, detección de fallos y comparativa de dashboards

FLUJO CRÍTICO EVALUADO EN LA TIENDA WEB:

1. **Inicio de Sesión (Login)**
2. **Catálogo de Galletas**
3. **Carrito de Compras**
4. **Procesamiento de Pago**

DEMO EN VIVO CON JMETER

- Simulación: De 50 a 100 usuarios comprando al mismo tiempo en el servidor local XAMPP.
- Métricas clave observadas:
  - Latencia y tiempo de respuesta en ms.
  - Consumo de CPU y memoria RAM.
  - Tasa de errores HTTP (404, 500).
- Conclusión de la demo: Se evidencia la estabilidad del sistema bajo alta demanda comercial.

COMPARATIVA DE DASHBOARD

Escenario A: Versión con fallos detectados

Tasa de error > 15%, cuellos de botella en MySQL y caídas de peticiones HTTP.

Escenario B: Versión optimizada con pipeline

0% errores, respuestas sub-segundo y pruebas automatizadas aprobadas con éxito.

ESTRATEGIAS CI/CD

Estrategias Avanzadas e Integración Continua

Metodologías profesionales para garantizar estabilidad, velocidad y cero fallos

Shift-Left Testing — Pruebas Tempranas	Probar desde el día 1 de desarrollo, no al final. Evita sorpresas antes de entregar.
Paralelización de Pruebas — Ahorro de Tiempo	Ejecutar suites de pruebas unitarias y de APIs simultáneamente en varios runners.
Entornos Aislados (Staging) — Seguridad Operativa	Nunca probar en producción en horas pico. Se despliega en réplicas idénticas antes.
Quality Gates (Puertas Lógicas) — Control Automático	Si una prueba falla o la cobertura baja de 80%, el pipeline aborta el despliegue.
Menor Time-to-Market — Impacto Comercial	De días de pruebas manuales a minutos: Galletas Natys publica ofertas al instante.

REGLA DE ORO: "Automatiza lo repetitivo y deja a las personas lo que requiere criterio."

ESTRATEGIAS CI/CD

Estrategias de Despliegue en Producción

Técnicas profesionales para actualizar sin interrupción del servicio (Zero-Downtime)

A. Despliegue Blue/Green (Azul / Verde)

CERO DOWNTIME

Dos entornos idénticos en producción: Azul (activo atendiendo clientes) y Verde (nueva versión probada en staging). Cuando se valida el Verde, el balanceador redirige el tráfico en 1 segundo. Si ocurre un fallo, se revierte al Azul de inmediato.

B. Despliegue Canario (Canary Releases)

LANZAMIENTO PROGRESIVO

La actualización se publica solo a un pequeño porcentaje de usuarios (ej. 5% al 10% de clientes de Galletas Natys). Se monitorean errores y latencia en vivo; si todo opera sin problemas, se libera gradualmente al 100% de la plataforma.

C. Despliegue Rolling (Progresivo)

NODO POR NODO

El sistema actualiza los servidores o instancias uno por uno o en pequeños lotes. Mientras unos nodos se actualizan, los demás continúan atendiendo peticiones, garantizando que la tienda web nunca sufra caídas.

CIERRE Y PARTICIPACIÓN ACTIVA

¡Escanea, Evalúa y Preguntas & Respuestas!

Dinámica interactiva con formulario en vivo y conclusiones finales

¡ESCANEA Y PARTICIPA!

Responde la evaluación de 19 preguntas desde tu celular y conoce tu nota al instante:

forms.gle/1q3Zo8qu2FE4vGpj9

CONCLUSIONES CLAVE

1. La automatización no reemplaza el criterio humano, optimiza el tiempo del equipo.

2. Los pipelines CI/CD eliminan los despliegues manuales propensos a fallos.

3. Apache JMeter garantiza que Galletas Natys resista picos de ventas reales.

4. Diseñar scripts bajo el patrón AAA permite mantenimiento escalable.

¿PREGUNTAS O COMENTARIOS?
¡Muchas gracias por su atención!

ANEXO 1 ·Mapa Conceptual

Automatización de Pruebas y Pipelines

Diagrama conceptual general del sistema integrado Galletas Natys



ESTRUCTURA DEL MAPA:

1. Fundamentos: Concepto, Manual vs. Auto, ROI y Pirámide.

2. Scripts de Prueba: Estructura 4 fases, Modularidad, POM y DDT.

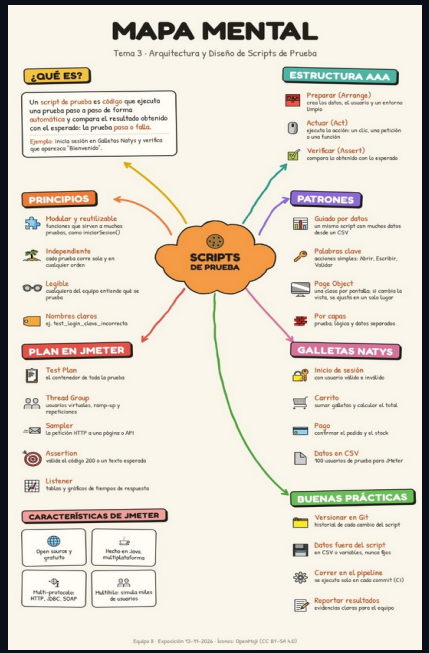
3. Pipelines CI/CD: Tuberías, etapas Build-Test-Deploy, Triggers y DevOps.

4. Estrategias Avanzadas: Shift-Left, Quality Gates, Blue/Green y Canary.

ANEXO 2 · Mapa Mental

Arquitectura y Diseño de Scripts de Prueba

Diagrama mental detallado: estructura AAA, principios, patrones y plan en JMeter



DESGLOSE DE RAMAS DEL MAPA MENTAL:

- Estructura AAA: Preparar (datos/entorno), Actuar (ejecutar acción) y Verificar (aserciones).
- Principios de Diseño: Modular y reutilizable, Independiente, Legible y Nombres claros.
- Patrones Profesionales: Data-Driven Testing (CSV), Page Object Model (POM) y separación por capas.
- Plan en Apache JMeter: Test Plan, Thread Group (hilos), Sampler (HTTP), Assertion y Listeners.
- Caso Galletas Natys: Inicio de sesión válido/inválido, Carrito y cálculo de total, Pago y stock.
- Buenas Prácticas: Versionar en Git, datos fuera del script, correr en CI y generar reportes.